

Proceso de Desarrollo de Software - Parcial 2 Ejemplo

Ejercicio 1

Una empresa de logística de vanguardia está desarrollando su nuevo Módulo de Planificación de Envíos. El sistema debe gestionar paquetes individuales y contenedores combinados, calcular costos de envío con tasas variables y centralizar la creación de las entidades para evitar el acoplamiento directo del cliente.

El negocio tiene las siguientes reglas de análisis funcional:

Una carga puede ser un `PaqueteSimple` (tiene un peso en kg y un costo base de envío fijo) o un `ContenedorCompuesto` (un conjunto de cargas que puede agrupar tanto paquetes simples como otros contenedores compuestos de manera indefinida). El costo de un `ContenedorCompuesto` se calcula sumando el costo de todas las cargas que tiene en su interior.

A cualquier tipo de carga (sea un paquete o un contenedor completo) se le pueden añadir dinámicamente factores que alteran su costo final:

- `EnvíoPrioritario`: Añade un costo fijo de \$1500 por el tratamiento urgente.
- `RefrigeraciónCadenaDeFrio`: Multiplica el costo total acumulado de la carga que envuelve por \$1.25\$ (un 25% de recargo).

El cliente (el sistema de facturación externo) no tiene permitido utilizar la palabra clave `new` para instanciar los paquetes ni los contenedores. Se debe disponer de una clase centralizada con un método estático que, según un parámetro o enumerado (`FormatoCarga`), devuelva la instancia limpia de la carga correspondiente lista para ser procesada.

Consigna: Diseñe el diagrama de clases completo, la solución debe fusionar de forma arquitectónicamente correcta los patrones de diseño que elija utilizar respetando el polimorfismo del sistema. Debajo del diagrama justifique la elección de cada uno de los patrones.

Ejercicio 2

Un desarrollador intentó implementar el ciclo de vida de un Envío que pasa por los estados Creado, EnViaje, Entregado y Cancelado. El código actual compila, pero tiene errores conceptuales graves:

Java

```
// Interfaz del Estado
interface EstadoPedido {
    void procesarEnvio(PedidoContext contexto);
    void cancelar(PedidoContext contexto);
}
```

```
// Clase Contexto
public class PedidoContext {
    public EstadoPedido estadoActual;
    private String codigoSeguridad;

    public PedidoContext(String codigo) {
        this.codigoSeguridad = codigo;
        // El pedido arranca en estado Creado
        this.estadoActual = new EstadoCreado();
    }

    public void ejecutarOperacion() {}

    public void cancelarPedido() {}

    public void setEstado(EstadoPedido nuevoEstado) {
        this.estadoActual = nuevoEstado;
    }
}

// Estado Concreto
class EstadoCreado implements EstadoPedido {
    @Override
    public void procesarEnvio(PedidoContext contexto) {
        System.out.println("Validando datos del pedido...");
    }

    @Override
    public void cancelar(PedidoContext contexto) {
        System.out.println("Pedido cancelado con éxito.");
    }
}
```

Consigna:

1. Identifique el patrón que se intenta implementar.
2. Identifique los errores conceptuales de diseño en esta implementación que violan las buenas prácticas explicadas en clase.
3. Reescriba el fragmento de código en Java corrigiendo los errores y completando las secciones correspondientes para que el Contexto delegue correctamente y el Estado Concreto transicione al siguiente de manera segura.

Ejercicio 3

Analice la siguiente propuesta de código para un sistema donde una EstacionMaritima les avisa a los barcos cargueros cuando cambia la altura de la marea. El desarrollador afirma haber aplicado un patrón de diseño.

Java

```
public class EstacionMaritima {
    private float alturaMarea;
    private List<Buque> barcosRegistrados = new ArrayList<>();

    public void registrarBarco(Buque b) {
        barcosRegistrados.add(b);
    }
}
```

```
    }

    public void setMarea(float nuevaAltura) {
        this.alturaMarea = nuevaAltura;
        for (Buque b: barcosRegistrados) {
            b.notificarCambioMarea(this.alturaMarea);
        }
    }
}
```

Consigna:

1. Especifique el patrón de diseño que se intentó implementar.
2. Explique detalladamente por qué esta solución no cumple con las propiedades fundamentales del patrón y las buenas prácticas de programación vistas en la materia.
3. Reescriba el fragmento de código en Java corrigiendo los errores